

ISE 503: Health Analytics
Project — Brief Interim Report
Predicting High-Cost Medicare Patients Before They Become High-Cost
Vansh Chanchlani & Sagarika Raju | vchanchl@usc.edu | rajus@usc.edu

1. Introduction

Healthcare cost concentration is one of the central challenges in U.S. health policy. Roughly **5% of Medicare beneficiaries account for approximately 50% of total Medicare expenditures** each year (Cohen & Yu, 2012). Despite this well-documented skewness, health systems and Accountable Care Organizations (ACOs) operating under value-based payment arrangements typically identify high-cost patients *only after* expenditures have already been incurred, leaving no actionable window for proactive intervention.

This pattern is particularly consequential under risk-based contracts, where providers bear direct financial responsibility for avoidable costs. Early identification of patients likely to enter the high-cost tier in the following year would enable targeted care management strategies nurse outreach, medication adherence programs, preventive visits, and care coordination that can meaningfully reduce avoidable hospitalizations and generate measurable savings. Predictive analytics, applied to longitudinal Medicare claims data, offers a principled and scalable approach to this challenge.

2. Health Analytics Framework

Objective

To build and evaluate a binary classification model that identifies Medicare beneficiaries at risk of falling in the top 5% of total annual spending in the following year, using only prior-year claims history, enabling proactive care management interventions.

Research Question

Can we accurately predict which Medicare beneficiaries will become high-cost patients (top 5% of spending) in Year 2 using patient-level features derived solely from their Year 1 claims data?

Target Population

Fee-for-service Medicare beneficiaries with at least one year of claims history, including inpatient, outpatient, and carrier claims. Beneficiaries with fewer than 12 months of Part A and Part B enrollment in Year 1 are excluded to ensure complete prior-year exposure.

Intended Audience

- Health system analytics and population health management teams
- ACO medical directors and care coordination staff
- CMS and policymakers evaluating value-based care program design

Decision-Making Problem and Use Case

The primary use case is prospective risk stratification: generate a ranked list of beneficiaries predicted to be high-cost in the coming year, enabling care management teams to allocate limited outreach resources toward those with the highest predicted risk and the greatest potential for cost-reducing intervention.

Primary Evaluation Focus

The project prioritizes both predictive accuracy and interpretability. Accuracy is critical for operational utility (correctly flagging the right patients), while interpretability achieved through SHAP-based explanations is essential for clinical trust and actionable insight into which patient characteristics drive predictions. Primary metrics are AUC-ROC, Precision at top-K, and Recall.

Problem Framing / Boundaries

- Binary classification: top-5% spending in Year 2 (label = 1) vs. all others (label = 0)
- Features drawn exclusively from Year 1 claims; no Year 2 data used in model training
- Scope limited to fee-for-service Medicare; Medicare Advantage excluded due to claims incompleteness
- Cost reduction interventions themselves are out of scope; the project focuses on identification only

3. Analytical Approach and Data

Dataset

We use the **CMS Synthetic Medicare Public Use Files (PUFs)** (<https://data.cms.gov/>), a publicly available, de-identified dataset produced by the Centers for Medicare and Medicaid Services. The dataset contains beneficiary-level demographics, enrollment information, inpatient claims, outpatient claims, and carrier (physician) claims across 8,671 beneficiaries in 2022 and 9,179 beneficiaries in 2023. This is a verified government source with the same structure as real CMS Research Identifiable Files, making it appropriate for academic modeling and methodology development.

Data Structure and Linkage

Year 1 features are constructed from the 2022 Beneficiary Summary File merged with 2022 inpatient, outpatient, and carrier claims. Year 2 labels are derived from 2023 total spending. Files are linked via BENE_ID. After merging and filtering for beneficiaries present in both years, the analytical dataset contains 3,339 beneficiaries with no missing values.

Feature Engineering

Patient-level features constructed from Year 1 claims include: (1) total annual spending and spending by service category (inpatient, outpatient, carrier); (2) inpatient hospitalization frequency and total utilization days; (3) outpatient visit count and carrier claim count; and (4) demographic variables including age, sex, race, and dual-eligibility months. The binary outcome label (top-5% spender in Year 2) is derived from the Year 2 spending distribution, with a threshold of \$85,798.

Planned Model Building Pipeline

- Data preprocessing: merge beneficiary summary and claims files; aggregate spending and utilization; handle class imbalance via SMOTE oversampling
- Baseline model: Logistic Regression with L2 regularization for interpretability and benchmarking
- Primary model: XGBoost gradient boosting for strong performance on tabular, imbalanced healthcare data
- Explainability: SHAP values applied to the XGBoost model to identify the most influential predictors
- Evaluation: AUC-ROC, Precision at top-K, and Recall at a fixed threshold
- Benchmarking: model performance compared against published results from Chechulin et al. (2014) and Rose (2016)

Expected Contribution

This project will produce an interpretable, operationally deployable risk stratification model grounded in publicly available Medicare data. SHAP analysis will surface clinically meaningful predictors — expected to include prior spending, hospitalization frequency, and demographic risk factors — that care management teams can act upon. The framework is designed to generalize to other payer populations with analogous claims structures.

4. Initial Results and Next Steps

Exploratory Data Analysis

Spending distribution. Year 2 total Medicare payments are highly right-skewed, with the majority of beneficiaries clustering near zero and a small proportion incurring extremely high costs (Figure 1). The empirical top-5% spending threshold is \$85,798, confirming the well-documented concentration of Medicare costs. On a log scale, the distribution approximates normality, validating the modeling approach.

Prior-year spending as a predictor. Future high-cost patients exhibit dramatically higher Year 1 spending (\$1,160.8K mean) compared to non-high-cost patients (\$88.4K mean) — a 13x differential (Figure 2). This provides strong initial evidence that prior-year claims history is predictive of future high-cost status, consistent with published literature.

Demographic patterns. High-cost patients skew slightly younger than non-high-cost patients, with a density peak around age 65 compared to age 70 for non-high-cost beneficiaries (Figure 3a). Males exhibit a higher high-cost rate (5.8%) compared to females (4.3%) (Figure 3b). Dual eligibility showed no variation in the current dataset, suggesting this variable may be limited in discriminatory power for this sample.

Utilization patterns. High-cost patients show markedly higher inpatient utilization in Year 1, with a median of 2 inpatient claims compared to 1 for non-high-cost patients (Figure 4a). Outpatient and carrier claim patterns are more variable, with high-cost patients showing wider interquartile ranges, indicating heterogeneous care pathways among this population.

Initial Modeling Results

A Logistic Regression baseline model with L2 regularization was trained on Year 1 features using SMOTE oversampling to address the 5:95 class imbalance. The model achieved an **AUC-ROC of 0.828** on the held-out test set (30% of data), well above the random baseline of 0.5 and comparable to published benchmarks for high-cost patient prediction (Chechulin et al., 2014; Rose, 2016). The model achieved a recall of 0.64 for high-cost patients, meaning it correctly identifies 64% of future high-cost patients using only prior-year claims data (Figure 5). Full XGBoost modeling with SHAP analysis is planned as the next step.

Next Steps

- Implement XGBoost gradient boosting with cross-validation and hyperparameter tuning
- Run SHAP analysis on the trained XGBoost model to identify global and patient-level feature importance
- Evaluate models on Precision-Recall curves and Precision at top-K for operational care management use case
- Benchmark final model performance against Chechulin et al. (2014) and Rose (2016)
- Develop final report, executive summary, and 7-slide presentation

5. Challenges

Identification of Challenges

Class imbalance. The top-5% label creates a 5:95 class imbalance. We address this with SMOTE oversampling and will evaluate models on Precision-Recall curves rather than accuracy alone.

Limited spending columns in beneficiary summary. The CMS Synthetic PUF beneficiary summary files do not include pre-aggregated annual spending totals. We derived total spending by aggregating CLM_PMT_AMT across inpatient, outpatient, and carrier claims files, which required careful multi-file merging and year filtering.

Sample size after merging. After restricting to beneficiaries present in both Year 1 (2022) and Year 2 (2023) claims, the analytical dataset contains 3,339 patients. While sufficient for modeling, this limits statistical power for subgroup analyses.

Interpretability for non-technical audiences. SHAP explanations must be translated from statistical output into language relevant to clinicians and care managers. We will frame findings around actionable clinical constructs — for example, identifying which specific utilization patterns most strongly drive high-cost predictions.

References

Centers for Medicare and Medicaid Services. (2023). Medicare public use files. U.S. Department of Health and Human Services. <https://data.cms.gov/>

Chechulin, Y., Nazerian, A., Rais, S., & Malikov, K. (2014). Predicting patients with high risk of becoming high-cost healthcare users in Ontario (Canada). *Healthcare Policy*, 9(3), 68–79. <https://doi.org/10.12927/hcpol.2014.23710>

Cohen, S. B., & Yu, W. (2012). The concentration and persistence in the level of health expenditures over time: Estimates for the U.S. population, 2008–2009 (Statistical Brief No. 354). Agency for Healthcare Research and Quality.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.

Quan, H., Sundararajan, V., Halfon, P., Fong, A., Burnand, B., Luthi, J. C., Saunders, L. D., Beck, C. A., Feasby, T. E., & Ghali, W. A. (2005). Coding algorithms for defining comorbidities in ICD-9-CM and ICD-10 administrative data. *Medical Care*, 43(11), 1130–1139. <https://doi.org/10.1097/01.mlr.0000182534.19832.83>

Rose, S. (2016). A machine learning framework for plan payment normalization. *American Journal of Health Economics*, 2(2), 207–232. https://doi.org/10.1162/AJHE_a_00040

Appendix A: Figures and Tables

Figure 1: Year 2 Medicare Spending Distribution

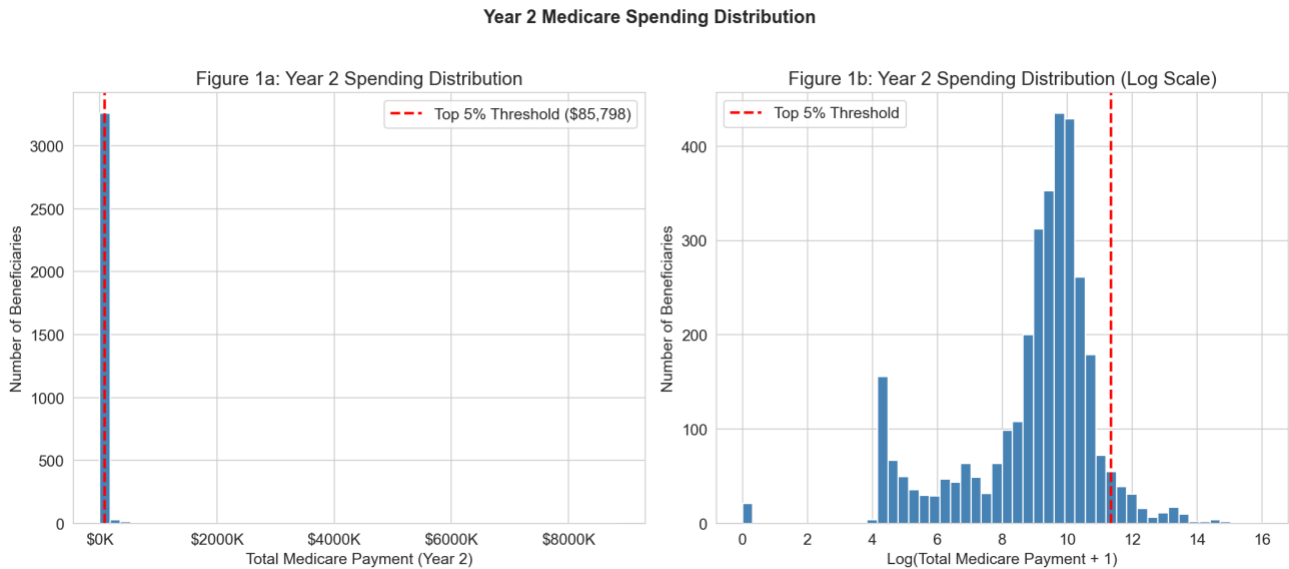
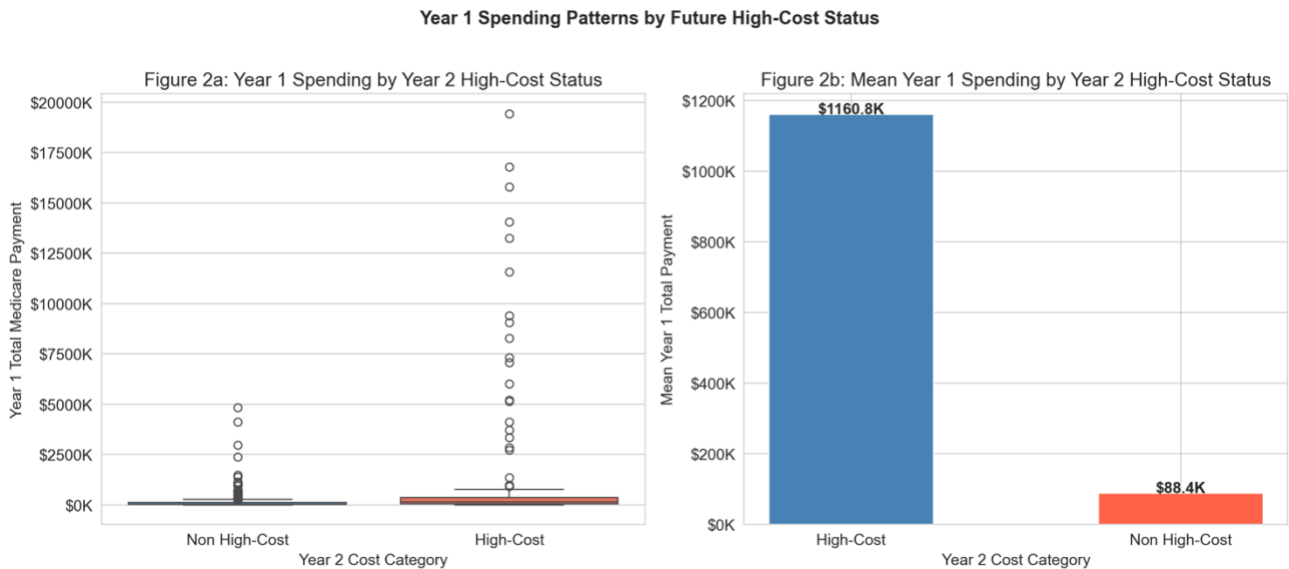


Figure 1a shows the raw Year 2 spending distribution across 3,339 beneficiaries, confirming extreme right skew with the top-5% threshold marked at \$85,798. Figure 1b presents the same distribution on a log scale, revealing an approximately normal underlying structure. These patterns are consistent with the well-documented concentration of Medicare expenditures (Cohen & Yu, 2012).

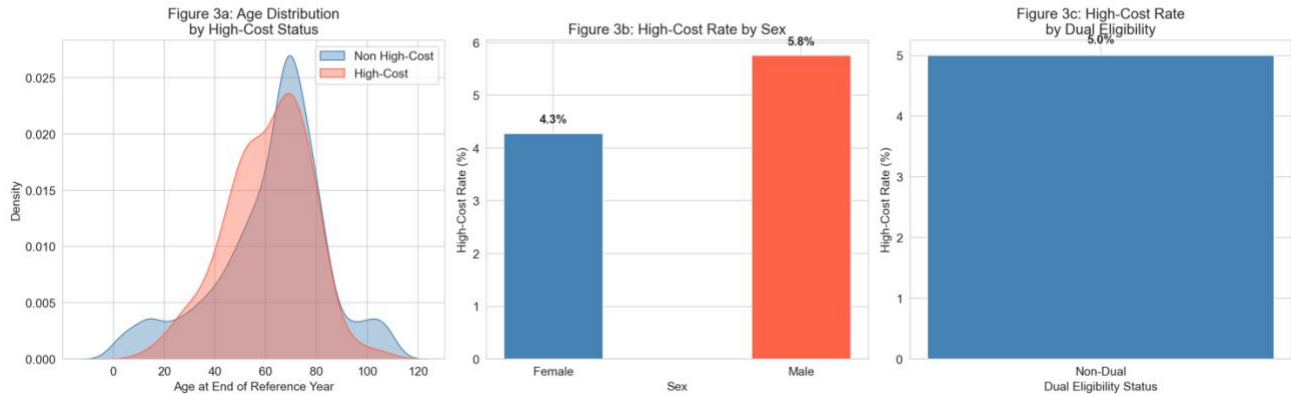
Figure 2: Year 1 Spending by Future High-Cost Status



Future high-cost patients (Year 2) exhibit a mean Year 1 total Medicare payment of \$1,160.8K compared to \$88.4K for non-high-cost patients — a 13x differential. This strongly validates prior-year spending as a key predictive feature.

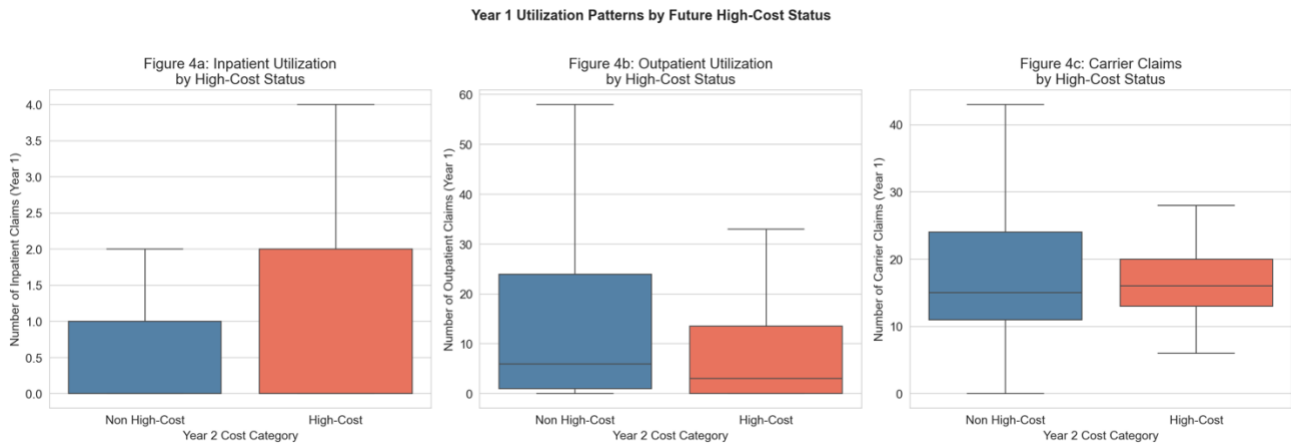
Figure 3: Demographic Analysis

Demographic Analysis of High-Cost Medicare Patients



High-cost patients skew slightly younger (peak ~65) than non-high-cost patients (peak ~70). Males exhibit a higher high-cost rate (5.8%) versus females (4.3%). Dual eligibility showed no variation in the current dataset sample.

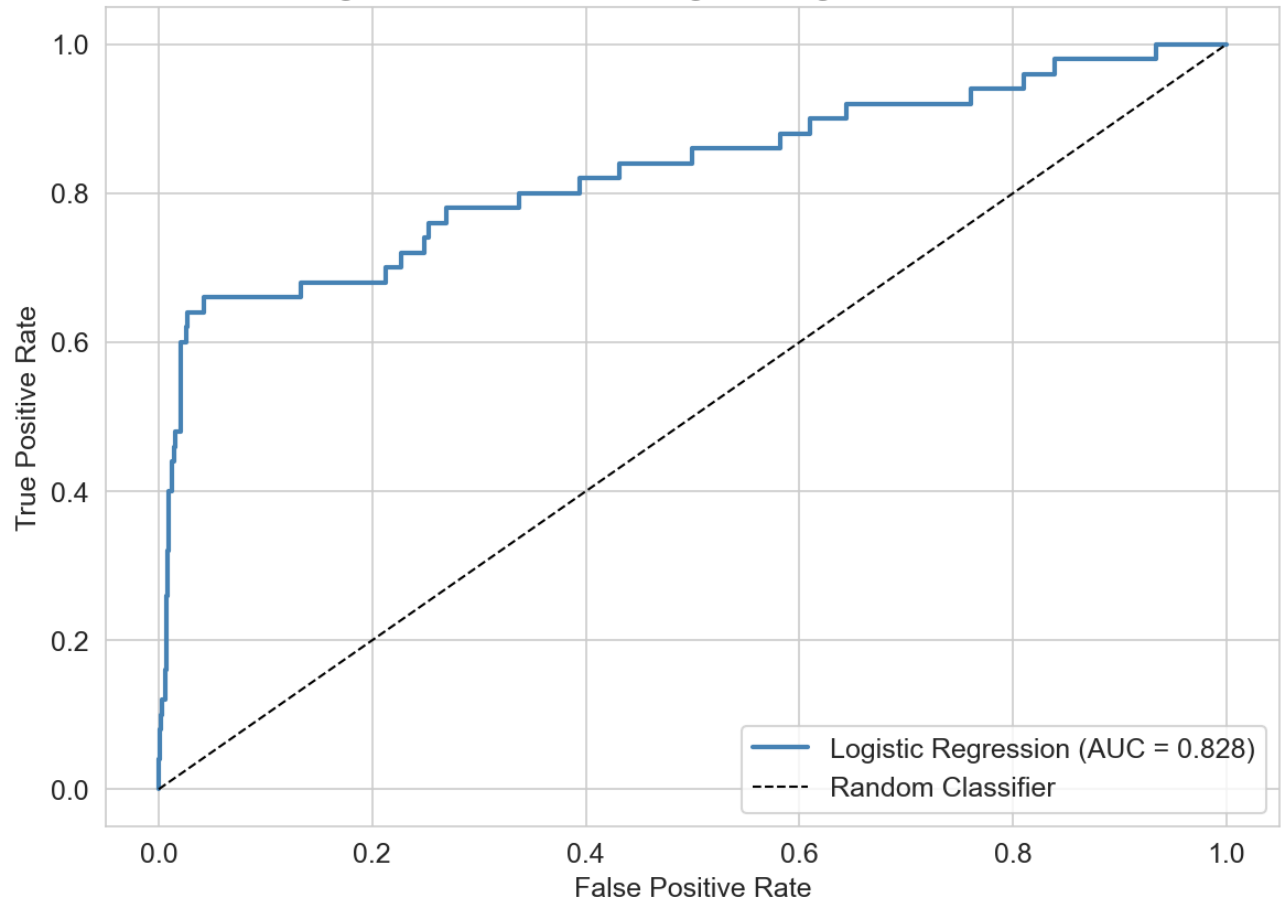
Figure 4: Year 1 Utilization Patterns by Future High-Cost Status



High-cost patients show higher inpatient utilization (median 2 claims vs 1) and wider interquartile ranges across outpatient and carrier claims, indicating more intense and variable healthcare use in the year preceding high-cost status.

Figure 5: ROC Curve — Logistic Regression Baseline (AUC = 0.828)

Figure 5: ROC Curve - Logistic Regression Baseline



The baseline Logistic Regression model achieves an AUC-ROC of 0.828, demonstrating strong discrimination between future high-cost and non-high-cost patients using only Year 1 claims features. The model correctly identifies 64% of future high-cost patients at the default classification threshold.

Note: All figures are generated from the Python analysis notebook (Appendix B) and saved to the project outputs directory.

Appendix B: Code

The full analysis was conducted in Python using a Jupyter Notebook (`medicare_project.ipynb`). Key libraries used: pandas, numpy, matplotlib, seaborn, scikit-learn, imbalanced-learn, xgboost, and shap. The notebook is structured as follows:

- Cell 1: Library imports and plot styling
- Cell 2: Load beneficiary (2022, 2023) and claims files (inpatient, outpatient, carrier)
- Cell 3: Aggregate spending by year; compute total payments per beneficiary
- Cell 4: Create binary high-cost label from Year 2 top-5% spending threshold (\$85,798)
- Cell 5: Merge Year 1 features with Year 2 labels; final dataset: 3,339 beneficiaries, 14 features, 0 missing values
- Cell 6: EDA — spending distribution plots (Figure 1)
- Cell 7: EDA — Year 1 spending by high-cost label (Figure 2)
- Cell 8: EDA — demographic analysis by high-cost status (Figure 3)
- Cell 9: EDA — utilization patterns by high-cost status (Figure 4)
- Cell 10: Baseline Logistic Regression with SMOTE, AUC-ROC evaluation, ROC curve (Figure 5)

Full code is available in the project repository (*ipynb converted to pdf and attached at the end of the report*). The notebook is reproducible end-to-end using the CMS Synthetic Medicare PUF files downloaded from <https://data.cms.gov/>.

```

#
=
# ISE 503 - Health Analytics | Spring 2026
# Project: Predicting High-Cost Medicare Patients Before They Become
High-Cost
# Team: Vansh Chanchlani & Sagarika Raju
#
=

# — IMPORTS

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (roc_auc_score, classification_report,
                             roc_curve, precision_recall_curve,
                             average_precision_score,
                             confusion_matrix)
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import shap
import warnings
warnings.filterwarnings('ignore')

# Plot styling
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12
sns.set_style('whitegrid')
sns.set_palette('muted')

print("✓ All libraries loaded successfully")
✓ All libraries loaded successfully

# — LOAD BENEFICIARY FILES

bene_2022 = pd.read_csv('../datasets/beneficiary_2022.csv', sep='|')
bene_2023 = pd.read_csv('../datasets/beneficiary_2023.csv', sep='|')

print("Beneficiary 2022 shape:", bene_2022.shape)
print("Beneficiary 2023 shape:", bene_2023.shape)

# — LOAD CLAIMS FILES

```

```

print("\nLoading claims files...")

inp_full = pd.read_csv('../datasets/inpatient.csv', sep='|',
                        usecols=['BENE_ID', 'CLM_FROM_DT',
                                'CLM_PMT_AMT',
                                'CLM_TOT_CHRG_AMT',
                                'CLM_UTLZTN_DAY_CNT'])

out_full = pd.read_csv('../datasets/outpatient.csv', sep='|',
                        usecols=['BENE_ID', 'CLM_FROM_DT',
                                'CLM_PMT_AMT',
                                'CLM_TOT_CHRG_AMT'])

car_full = pd.read_csv('../datasets/carrier.csv', sep='|',
                        usecols=['BENE_ID', 'CLM_FROM_DT',
                                'CLM_PMT_AMT'])

# Parse dates and extract year
for df in [inp_full, out_full, car_full]:
    df['CLM_FROM_DT'] = pd.to_datetime(df['CLM_FROM_DT'],
    errors='coerce')
    df['YEAR'] = df['CLM_FROM_DT'].dt.year

print("✓ Inpatient:", inp_full.shape)
print("✓ Outpatient:", out_full.shape)
print("✓ Carrier:", car_full.shape)
print("\nClaims years available:",
sorted(inp_full['YEAR'].dropna().unique()))

Beneficiary 2022 shape: (8671, 185)
Beneficiary 2023 shape: (9179, 185)

Loading claims files...
✓ Inpatient: (58066, 6)
✓ Outpatient: (575092, 5)
✓ Carrier: (1121004, 4)

Claims years available: [2015, 2016, 2017, 2018, 2019, 2020, 2021,
2022, 2023]

# — BUILD YEAR 1 (2022) AND YEAR 2 (2023) SPENDING

```

```

def get_spending_by_year(inp, out, car, year):
    """Aggregate total Medicare payments per beneficiary for a given
    year."""

    inp_yr = inp[inp['YEAR'] == year].groupby('BENE_ID').agg(
        INP_PMT=('CLM_PMT_AMT', 'sum'),

```

```

        INP_STAYS=('CLM_PMT_AMT', 'count'),
        INP_DAYS=('CLM_UTLZTN_DAY_CNT', 'sum')
    ).reset_index()

    out_yr = out[out['YEAR'] == year].groupby('BENE_ID').agg(
        OUT_PMT=('CLM_PMT_AMT', 'sum'),
        OUT_VISITS=('CLM_PMT_AMT', 'count')
    ).reset_index()

    car_yr = car[car['YEAR'] == year].groupby('BENE_ID').agg(
        CAR_PMT=('CLM_PMT_AMT', 'sum'),
        CAR_CLAIMS=('CLM_PMT_AMT', 'count')
    ).reset_index()

    spending = inp_yr.merge(out_yr, on='BENE_ID', how='outer')\
        .merge(car_yr, on='BENE_ID', how='outer')
    spending = spending.fillna(0)
    spending['TOTAL_PMT'] = spending['INP_PMT'] + spending['OUT_PMT']
+ spending['CAR_PMT']
    return spending

# Build spending for Year 1 and Year 2
spend_2022 = get_spending_by_year(inp_full, out_full, car_full, 2022)
spend_2023 = get_spending_by_year(inp_full, out_full, car_full, 2023)

print("Year 1 (2022) beneficiaries with claims:", len(spend_2022))
print("Year 2 (2023) beneficiaries with claims:", len(spend_2023))

print("\nYear 1 spending summary:")
print(spend_2022['TOTAL_PMT'].describe().round(2))

print("\nYear 2 spending summary:")
print(spend_2023['TOTAL_PMT'].describe().round(2))

# — CREATE HIGH-COST LABEL FROM YEAR 2
threshold = spend_2023['TOTAL_PMT'].quantile(0.95)
spend_2023['HIGH_COST'] = (spend_2023['TOTAL_PMT'] >=
threshold).astype(int)

print(f"\nTop 5% spending threshold (Year 2): ${threshold:,.2f}")
print("High cost patients (1):", spend_2023['HIGH_COST'].sum())
print("Non high cost patients (0):",
(spend_2023['HIGH_COST']==0).sum())
print("\nClass distribution:")
print(spend_2023['HIGH_COST'].value_counts(normalize=True).round(3))

Year 1 (2022) beneficiaries with claims: 8301
Year 2 (2023) beneficiaries with claims: 3494

```

```
Year 1 spending summary:
count      8301.00
mean       85401.79
std        500934.23
min         0.00
25%        11223.50
50%        21719.90
75%        60013.50
max       19454109.68
Name: TOTAL_PMT, dtype: float64
```

```
Year 2 spending summary:
count      3494.00
mean       35375.67
std       197069.61
min         0.00
25%        4688.66
50%       13519.76
75%       24700.57
max      8864826.79
Name: TOTAL_PMT, dtype: float64
```

```
Top 5% spending threshold (Year 2): $85,797.90
High cost patients (1): 175
Non high cost patients (0): 3319
```

```
Class distribution:
HIGH_COST
0      0.95
1      0.05
Name: proportion, dtype: float64
```

— MERGE BENEFICIARY DEMOGRAPHICS WITH SPENDING

```
# Select useful demographic columns from beneficiary 2022
demo_cols = ['BENE_ID', 'AGE_AT_END_REF_YR', 'SEX_IDENT_CD',
             'BENE_RACE_CD',
             'ESRD_IND', 'DUAL_ELGBL_MONS', 'BENE_HI_CVRAGE_TOT_MONS',
             'BENE_SMI_CVRAGE_TOT_MONS', 'BENE_HMO_CVRAGE_TOT_MONS',
             'BENE_STATE_BUYIN_TOT_MONS', 'RTI_RACE_CD']

bene_demo = bene_2022[demo_cols].copy()

# Merge Year 1 spending with demographics
df_y1 = spend_2022.merge(bene_demo, on='BENE_ID', how='inner')

# Merge with Year 2 label
label = spend_2023[['BENE_ID', 'HIGH_COST', 'TOTAL_PMT']].rename(
    columns={'TOTAL_PMT': 'Y2_TOTAL_PMT'})
```

```

df_final = df_y1.merge(label, on='BENE_ID', how='inner')

print("Final dataset shape:", df_final.shape)
print("\nClass distribution in final dataset:")
print(df_final['HIGH_COST'].value_counts())
print("\nMissing values:")
print(df_final.isnull().sum()[df_final.isnull().sum() > 0])
print("\nSample:")
print(df_final.head(3).to_string())

```

Final dataset shape: (3339, 21)

Class distribution in final dataset:

HIGH_COST

0 3172

1 167

Name: count, dtype: int64

Missing values:

Series([], dtype: int64)

Sample:

	BENE_ID	INP_PMT	INP_STAYS	INP_DAYS	OUT_PMT	OUT_VISITS
CAR_PMT	CAR_CLAIMS	TOTAL_PMT	AGE_AT_END	REF_YR	SEX_IDENT_CD	
BENE_RACE_CD	ESRD_IND	DUAL_ELGBL_MONS	BENE_HI_CVRAGE_TOT_MONS			
BENE_SMI_CVRAGE_TOT_MONS	BENE_HMO_CVRAGE_TOT_MONS					
BENE_STATE	BUYIN_TOT_MONS	RTI_RACE_CD	HIGH_COST	Y2_TOTAL_PMT		
0	-10000010287996	0.00	0.0	0.0	125687.59	112.0
46881.82		23.0	172569.41		55	2
5	Y		0		12	
12			0		12	5
0		25150.04				
1	-10000010287992	166.97	1.0	1.0	1365.04	2.0
29496.80		16.0	31028.81		83	1
1	0		0		12	
12			0		12	1
0		682.52				
2	-10000010287978	166.97	1.0	1.0	81800.37	51.0
233805.04		156.0	315772.38		69	1
5	0		0		12	
12			0		12	5
0		24649.66				

— EDA: FIGURE 1 - YEAR 2 SPENDING DISTRIBUTION

```

import os
os.makedirs('../outputs', exist_ok=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

```

```

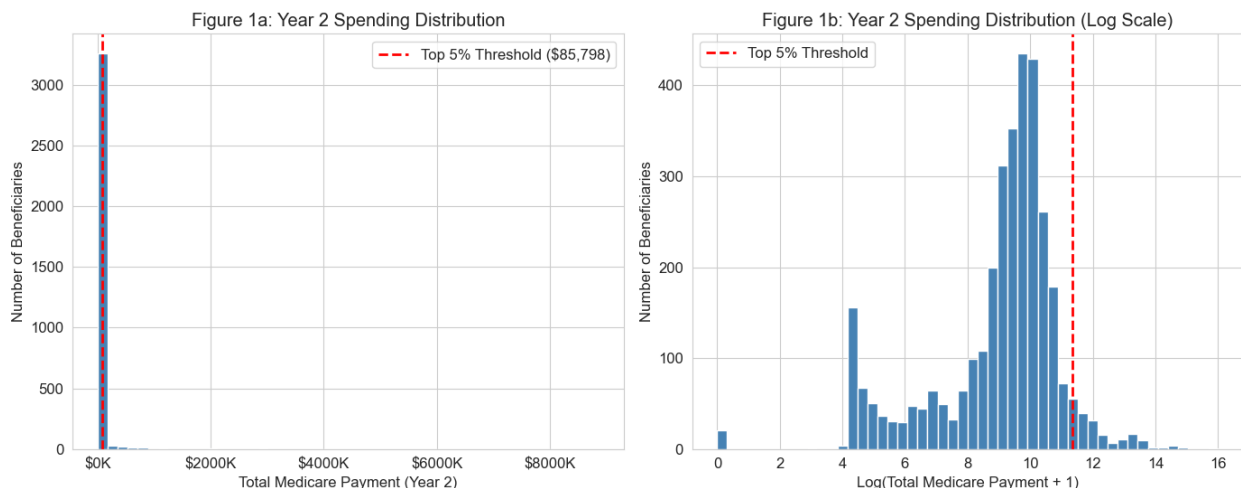
# Left: full distribution
axes[0].hist(df_final['Y2_TOTAL_PMT'], bins=50, color='steelblue',
edgecolor='white')
axes[0].axvline(df_final['Y2_TOTAL_PMT'].quantile(0.95),
                color='red', linestyle='--', linewidth=2, label='Top
5% Threshold ($85,798)')
axes[0].set_xlabel('Total Medicare Payment (Year 2)')
axes[0].set_ylabel('Number of Beneficiaries')
axes[0].set_title('Figure 1a: Year 2 Spending Distribution')
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1000:.0f}K'))
axes[0].legend()

# Right: log scale for better visibility
axes[1].hist(np.log1p(df_final['Y2_TOTAL_PMT']), bins=50,
color='steelblue', edgecolor='white')
axes[1].axvline(np.log1p(df_final['Y2_TOTAL_PMT'].quantile(0.95)),
                color='red', linestyle='--', linewidth=2, label='Top
5% Threshold')
axes[1].set_xlabel('Log(Total Medicare Payment + 1)')
axes[1].set_ylabel('Number of Beneficiaries')
axes[1].set_title('Figure 1b: Year 2 Spending Distribution (Log
Scale)')
axes[1].legend()

plt.suptitle('Year 2 Medicare Spending Distribution', fontsize=14,
fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure1_spending_distribution.png', dpi=150,
bbox_inches='tight')
plt.show()
print("✓ Figure 1 saved")

```

Year 2 Medicare Spending Distribution



✓ Figure 1 saved

— EDA: FIGURE 2 - YEAR 1 SPENDING BY HIGH-COST STATUS

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

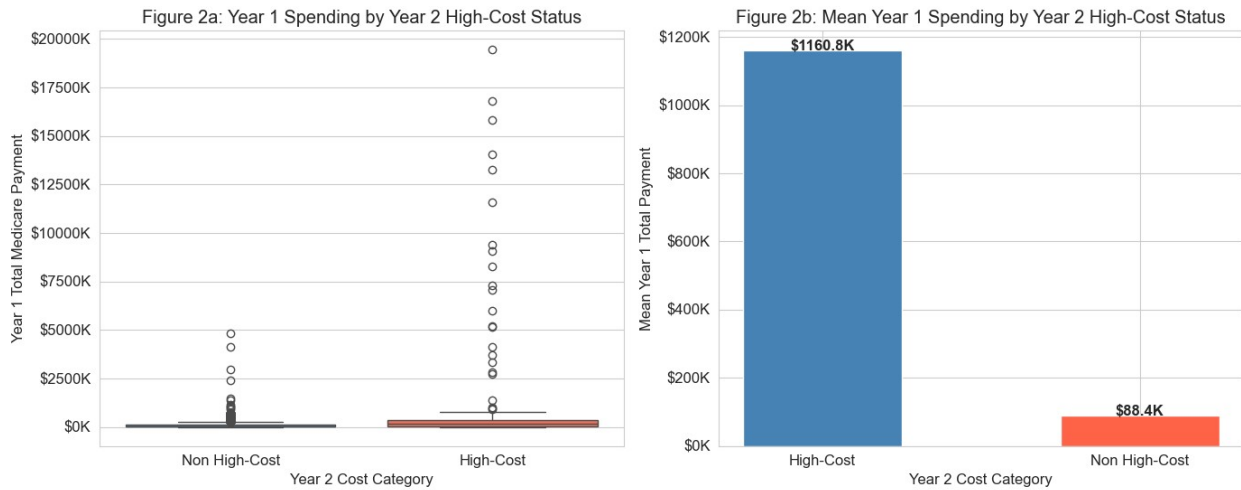
# Left: boxplot
df_final['HIGH_COST_LABEL'] = df_final['HIGH_COST'].map({0: 'Non High-
Cost', 1: 'High-Cost'})

sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='TOTAL_PMT',
            palette={'Non High-Cost': 'steelblue', 'High-Cost':
            'tomato'},
            ax=axes[0])
axes[0].set_xlabel('Year 2 Cost Category')
axes[0].set_ylabel('Year 1 Total Medicare Payment')
axes[0].set_title('Figure 2a: Year 1 Spending by Year 2 High-Cost
Status')
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1000:.0f}K'))

# Right: mean spending comparison bar chart
mean_spend = df_final.groupby('HIGH_COST_LABEL')
['TOTAL_PMT'].mean().reset_index()
bars = axes[1].bar(mean_spend['HIGH_COST_LABEL'],
mean_spend['TOTAL_PMT'],
                    color=['steelblue', 'tomato'], edgecolor='white',
width=0.5)
axes[1].set_xlabel('Year 2 Cost Category')
axes[1].set_ylabel('Mean Year 1 Total Payment')
axes[1].set_title('Figure 2b: Mean Year 1 Spending by Year 2 High-Cost
Status')
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1000:.0f}K'))
for bar, val in zip(bars, mean_spend['TOTAL_PMT']):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() +
1000,
                 f'${val/1000:.1f}K', ha='center', fontweight='bold')

plt.suptitle('Year 1 Spending Patterns by Future High-Cost Status',
            fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure2_spending_by_label.png', dpi=150,
bbox_inches='tight')
plt.show()
print("✓ Figure 2 saved")
```


Year 1 Spending Patterns by Future High-Cost Status



✓ Figure 2 saved

— EDA: FIGURE 3 - DEMOGRAPHIC ANALYSIS

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Left: Age distribution by high-cost status
sns.kdeplot(data=df_final[df_final['HIGH_COST']==0],
             x='AGE_AT_END_REF_YR', ax=axes[0],
             label='Non High-Cost', color='steelblue', fill=True,
             alpha=0.4)
sns.kdeplot(data=df_final[df_final['HIGH_COST']==1],
             x='AGE_AT_END_REF_YR', ax=axes[0],
             label='High-Cost', color='tomato', fill=True, alpha=0.4)
axes[0].set_xlabel('Age at End of Reference Year')
axes[0].set_ylabel('Density')
axes[0].set_title('Figure 3a: Age Distribution\nby High-Cost Status')
axes[0].legend()

# Middle: Sex breakdown
sex_map = {1: 'Male', 2: 'Female'}
df_final['SEX_LABEL'] = df_final['SEX_IDENT_CD'].map(sex_map)
sex_pct = df_final.groupby('SEX_LABEL')
['HIGH_COST'].mean().reset_index()
sex_pct.columns = ['Sex', 'High_Cost_Rate']
bars = axes[1].bar(sex_pct['Sex'], sex_pct['High_Cost_Rate']*100,
                  color=['steelblue', 'tomato'], edgecolor='white',
                  width=0.4)
axes[1].set_xlabel('Sex')
axes[1].set_ylabel('High-Cost Rate (%)')
axes[1].set_title('Figure 3b: High-Cost Rate by Sex')
for bar, val in zip(bars, sex_pct['High_Cost_Rate']):
    axes[1].text(bar.get_x() + bar.get_width()/2,
```

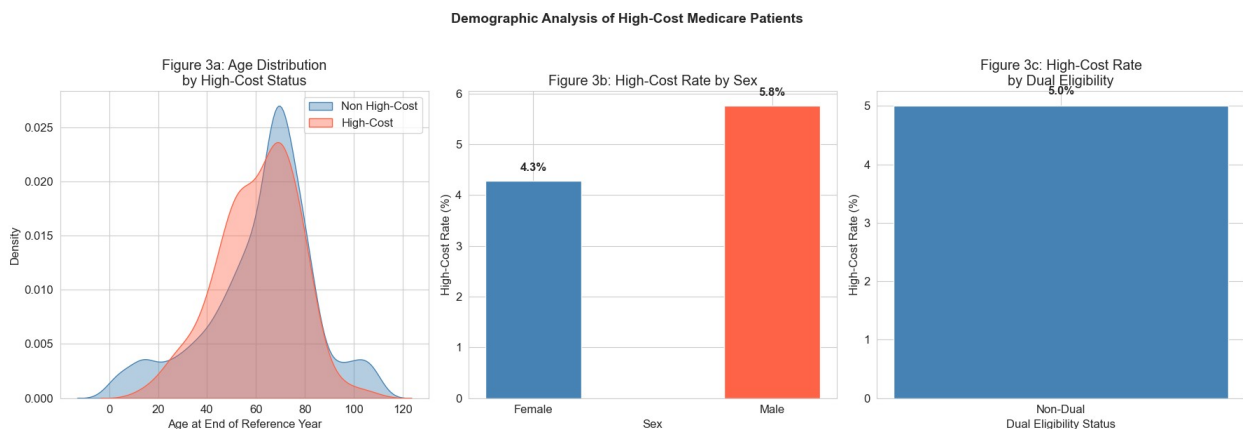
```

        bar.get_height() + 0.2,
        f'{val*100:.1f}%', ha='center', fontweight='bold')

# Right: Dual eligibility
df_final['DUAL_FLAG'] = (df_final['DUAL_ELGBL_MONS'] > 0).map(
    {True: 'Dual Eligible', False: 'Non-Dual'})
dual_pct = df_final.groupby('DUAL_FLAG')
['HIGH_COST'].mean().reset_index()
bars2 = axes[2].bar(dual_pct['DUAL_FLAG'], dual_pct['HIGH_COST']*100,
                    color=['steelblue', 'tomato'], edgecolor='white',
                    width=0.4)
axes[2].set_xlabel('Dual Eligibility Status')
axes[2].set_ylabel('High-Cost Rate (%)')
axes[2].set_title('Figure 3c: High-Cost Rate\nby Dual Eligibility')
for bar, val in zip(bars2, dual_pct['HIGH_COST']):
    axes[2].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.2,
                 f'{val*100:.1f}%', ha='center', fontweight='bold')

plt.suptitle('Demographic Analysis of High-Cost Medicare Patients',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure3_demographics.png', dpi=150,
            bbox_inches='tight')
plt.show()
print("✓ Figure 3 saved")

```



✓ Figure 3 saved

— EDA: FIGURE 4 - UTILIZATION PATTERNS

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
```

Left: Inpatient stays by high-cost status

```
sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='INP_STAYS',
            palette={'Non High-Cost': 'steelblue', 'High-Cost':
```

```

'tomato'},
    ax=axes[0], showfliers=False)
axes[0].set_xlabel('Year 2 Cost Category')
axes[0].set_ylabel('Number of Inpatient Claims (Year 1)')
axes[0].set_title('Figure 4a: Inpatient Utilization\nby High-Cost
Status')

# Middle: Outpatient visits
sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='OUT_VISITS',
    palette={'Non High-Cost': 'steelblue', 'High-Cost':
'tomato'},
    ax=axes[1], showfliers=False)
axes[1].set_xlabel('Year 2 Cost Category')
axes[1].set_ylabel('Number of Outpatient Claims (Year 1)')
axes[1].set_title('Figure 4b: Outpatient Utilization\nby High-Cost
Status')

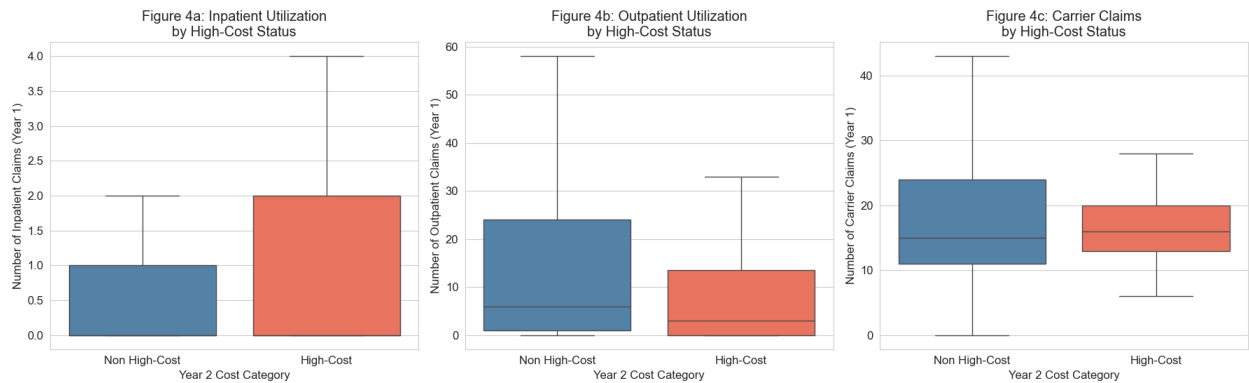
# Right: Carrier claims
sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='CAR_CLAIMS',
    palette={'Non High-Cost': 'steelblue', 'High-Cost':
'tomato'},
    ax=axes[2], showfliers=False)
axes[2].set_xlabel('Year 2 Cost Category')
axes[2].set_ylabel('Number of Carrier Claims (Year 1)')
axes[2].set_title('Figure 4c: Carrier Claims\nby High-Cost Status')

plt.suptitle('Year 1 Utilization Patterns by Future High-Cost Status',
    fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure4_utilization.png', dpi=150,
bbox_inches='tight')
plt.show()

# Print mean utilization stats
print("Mean utilization by high-cost status:")
print(df_final.groupby('HIGH_COST_LABEL')
[['INP_STAYS', 'OUT_VISITS', 'CAR_CLAIMS']].mean().round(1))
print("✓ Figure 4 saved")

```

Year 1 Utilization Patterns by Future High-Cost Status



Mean utilization by high-cost status:

	INP_STAYS	OUT_VISITS	CAR_CLAIMS
High-Cost	10.3	16.8	30.3
Non High-Cost	2.0	23.3	30.0

✓ Figure 4 saved

— CELL 9: BASELINE LOGISTIC REGRESSION

Feature columns

```
feature_cols = ['TOTAL_PMT', 'INP_PMT', 'OUT_PMT', 'CAR_PMT',
                'INP_STAYS', 'INP_DAYS', 'OUT_VISITS', 'CAR_CLAIMS',
                'AGE_AT_END_REF_YR', 'SEX_IDENT_CD', 'BENE_RACE_CD',
                'BENE_HI_CVRAGE_TOT_MONS', 'BENE_SMI_CVRAGE_TOT_MONS',
                'BENE_HMO_CVRAGE_TOT_MONS']
```

```
X = df_final[feature_cols]
y = df_final['HIGH_COST']
```

Train/test split (70/30, stratified)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

Scale features

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

Apply SMOTE to handle class imbalance

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train_sc, y_train)
print("After SMOTE - Class distribution:")
print(pd.Series(y_train_sm).value_counts())
```

Fit Logistic Regression

```
lr = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
```

```

lr.fit(X_train_sm, y_train_sm)

# Evaluate
y_pred_proba = lr.predict_proba(X_test_sc)[:, 1]
auc = roc_auc_score(y_test, y_pred_proba)
print(f"\n✓ Logistic Regression AUC-ROC: {auc:.4f}")

# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(fpr, tpr, color='steelblue', lw=2,
        label=f'Logistic Regression (AUC = {auc:.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random Classifier')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('Figure 5: ROC Curve - Logistic Regression Baseline')
ax.legend(loc='lower right')
plt.tight_layout()
plt.savefig('../outputs/figure5_roc_curve.png', dpi=150,
            bbox_inches='tight')
plt.show()

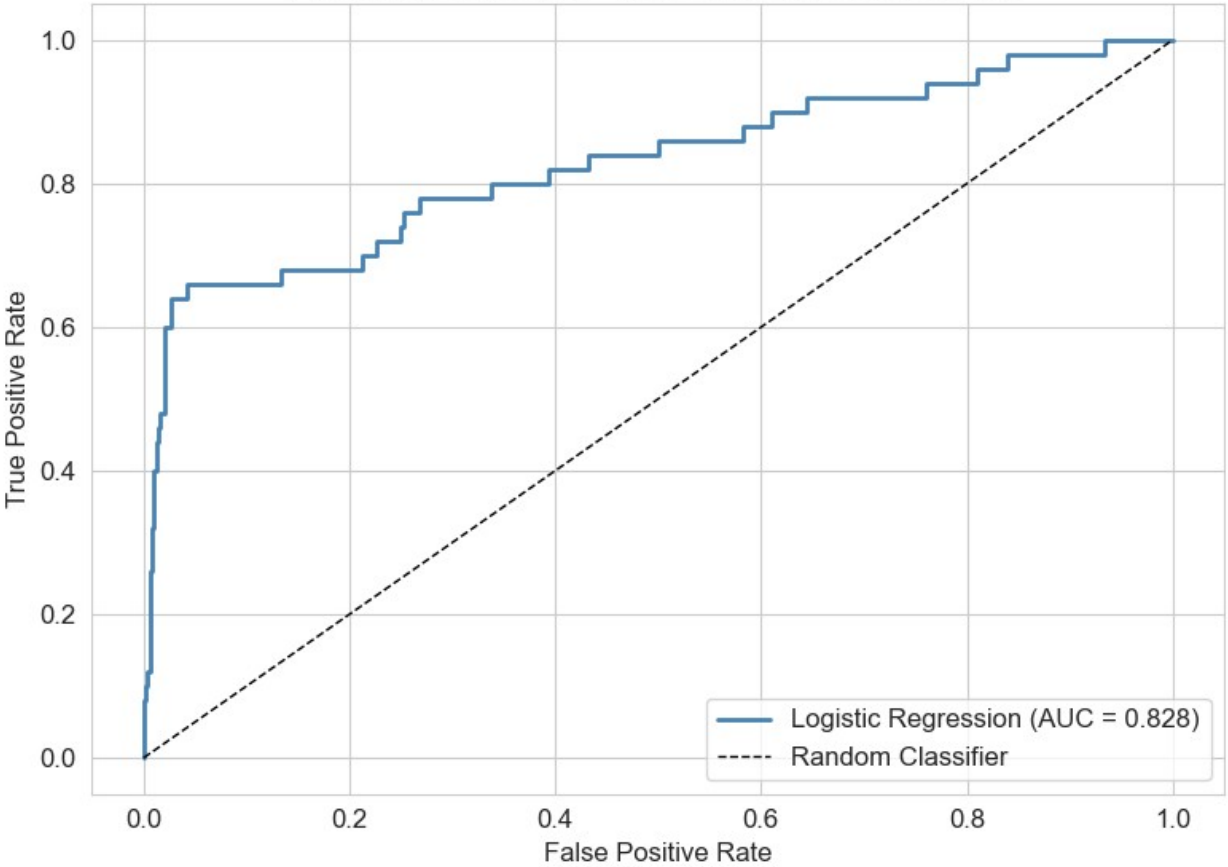
print("\nClassification Report:")
y_pred = lr.predict(X_test_sc)
print(classification_report(y_test, y_pred, target_names=['Non High-
Cost', 'High-Cost']))
print("✓ Figure 5 saved")

After SMOTE - Class distribution:
HIGH_COST
0      2220
1      2220
Name: count, dtype: int64

✓ Logistic Regression AUC-ROC: 0.8283

```

Figure 5: ROC Curve - Logistic Regression Baseline



Classification Report:

	precision	recall	f1-score	support
Non High-Cost	0.98	0.96	0.97	952
High-Cost	0.44	0.64	0.52	50
accuracy			0.94	1002
macro avg	0.71	0.80	0.75	1002
weighted avg	0.95	0.94	0.95	1002

✓ Figure 5 saved